

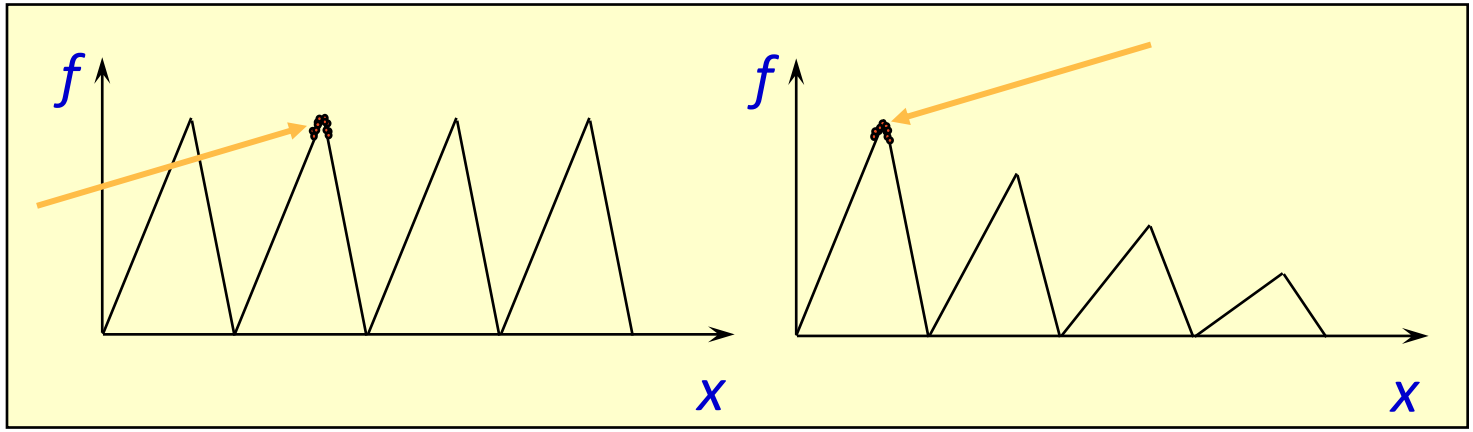
Not to be redistributed to Course Hero or any other public websites

DSAI6707 Advanced Computational Intelligence

Sem 1, 2026/2027

Multimodal Optimization

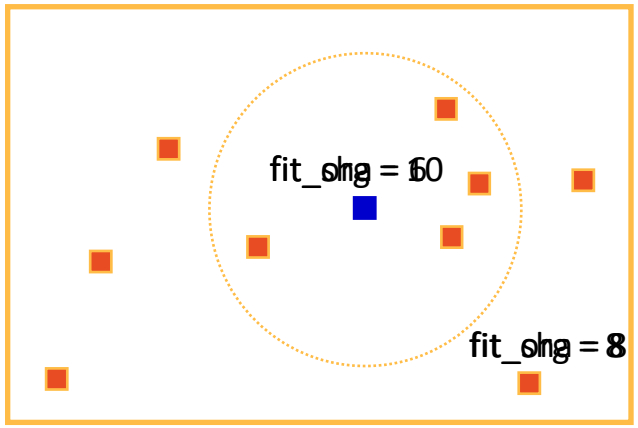
Problem: A simple genetic algorithm (GA) cannot maintain stable populations at different optima of functions with multiple, equal or unequal, peaks.



Solution: To introduce a controlled competition among different solutions near every locally optimal region, which is accomplished by incorporating the concepts of niche and species into GA.

- A simple GA with no *niching* converges to a single optimum even though multiple peaks of equal quality may exits. Nature addresses such a problem through the formation of stable subpopulations of organisms surrounding separate niches by forcing similar individuals to share their resources.

- *Niching* in general is achieved using a *sharing function*, which creates subdivisions of the environment *by degrading an organism's fitness proportional to the number of other members in its neighbourhood*.

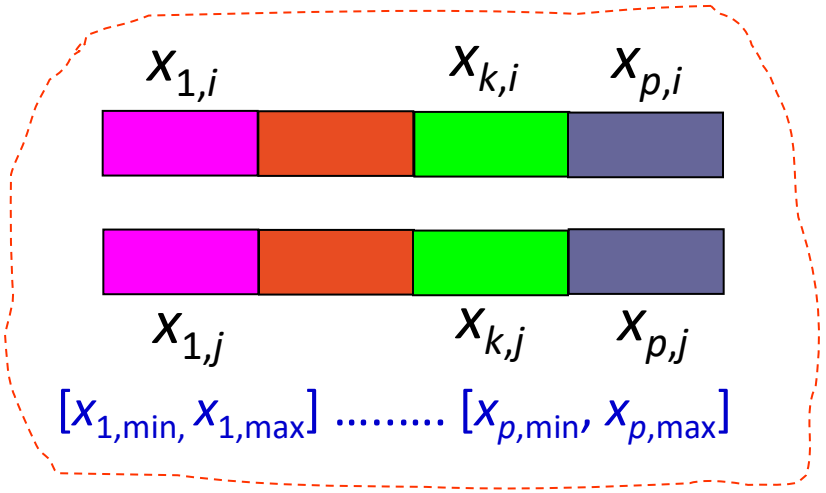


fit_sha is used in the selection process

Example: search space

- The amount of sharing contributed by each organism x_i into its neighbour x_j is determined by their *proximity* in the decoded parameter space based on distance measure d_{ij}. Given p parameters of unequal boundaries over a parameter range $[x_{min}, x_{max}]$

$$d_{ij} = \sqrt{\sum_{k=1}^{p} \left(\frac{x_{k,i} - x_{k,j}}{x_{k,\max} - x_{k,\min}} \right)^2}$$



$x_{k,i}$ = k-th parameter of individual i;

$x_{k,j}$ = k-th parameter of individual j;

$x_{k,\max}$ = Maximum allowable value for k-th parameter; and

$x_{k,\min}$ = Minimum allowable value for k-th parameter.

p = number of parameters

For each d_{ij}, we can apply the sharing function $s(d_{ij})$ given by

$$s(d_{ij}) = \begin{cases} 1 - \left(\frac{d_{ij}}{\sigma_{share}}\right)^{w} & \text{if } d_{ij} < \sigma_{share} \\ 0 & otherwise \end{cases}$$

s (i) penalty (i)
Fitness is reduced based on s

d (i) s (d)
s=1, d=0, if no indv inside lamda share

The limiting distance between the individuals to be shared, σ_{share} is calculated as the average distance required to identify each niche distinctly in the solution space.

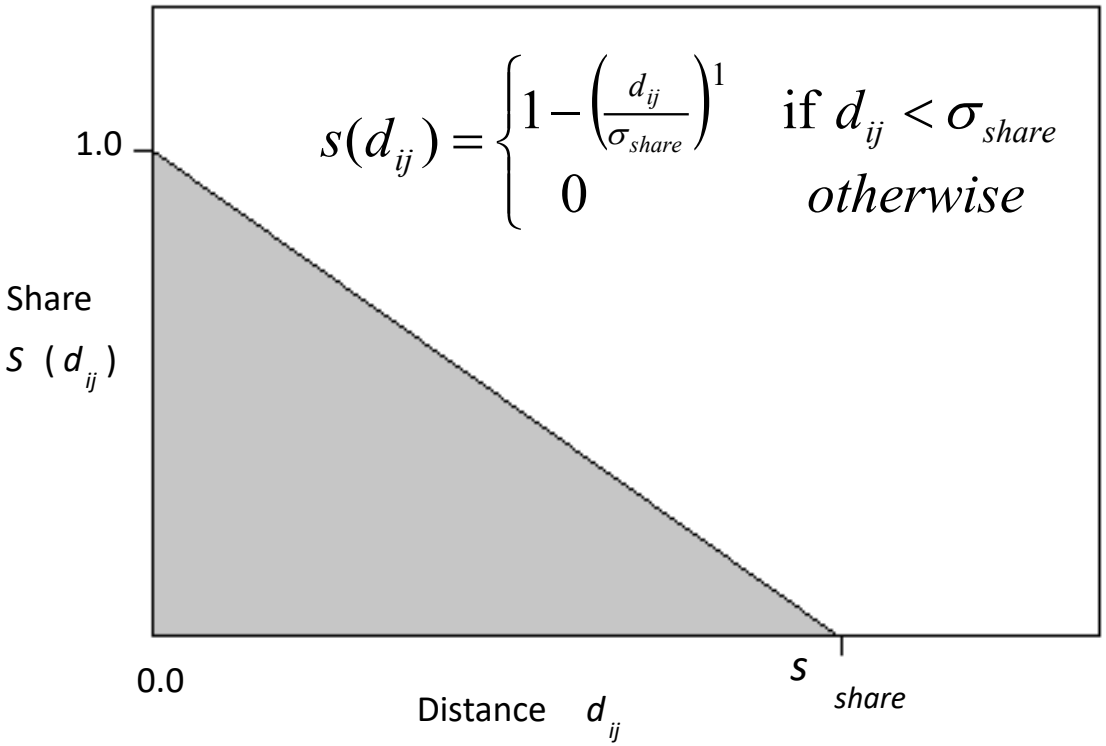
This value can be set based on q uniformly spaced assumed peaks,

q (i) lamda (d)
p (i) ss (i) lamda (i)

$$\sigma_{share} = 0.5q^{-1/p}$$

number of parameters

The triangular sharing function with $w = 1$ is used as it allows equal degree of sharing between the neighbouring individuals.



Triangular sharing function (Linear Sharing)

- $s(d_{ij}) \leq 1.0$ for all d_{ij}. There is a fractional contribution sharing effect when $0 < d_{ij} < \sigma_{share}$.
- $s(d_{ij}) = 1.0$. When both individuals are identical, they will share a full portion with each other.
- $\lim_{d_{ij} \to \infty} s(d_{ij}) \to 0$ when the individuals are separated $> \sigma_{share}$ or far apart, they produce no effect on each other.

The **shared fitness** of the *i*-th individual is given as

Original fitness obtained from cost function

Fitness used for selection process

$$SharedFitness = \frac{TrueFitness}{\sum_{j} s(d_{ij})}$$

min =1, s=1, d=0, no indv inside lamda share
Otherwise, fit_sha < fit_org

In a GA with niching/sharing, the niching/sharing algorithm is employed before the selection process.

Evolutionary Computation for Multi-objective Optimization

Rationale and Motivation

- Many real-world problems have several (possibly conflicting) objectives to be optimized
- Some of these problems are transformed (or aggregated) into a single-objective problem (SOP) and thus are solved using single objective optimization techniques
- Transforming multi-objective problem (MOP) into SOP requires priori knowledge of relative importance of different objectives
- Defining the problem in multi-objective framework is more general and it provides the decision maker different tradeoffs between different objectives
- The multi-objective optimization (also known as multi-criteria optimization) aims to find a set of vectors which satisfy the given constraints and optimizes a number of objective functions.

Example:

A two-objective functions, f_1 and f_2, to be minimized are given as

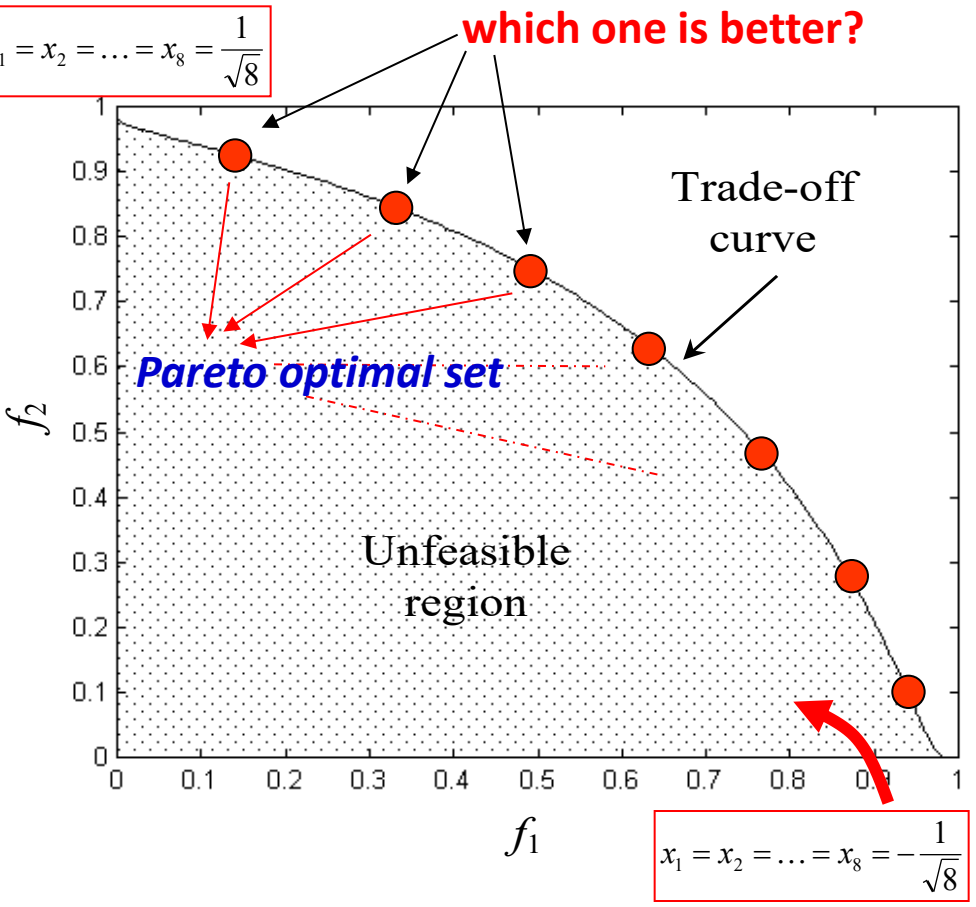
$$f_1(x_1,...,x_8) = 1 - exp\left(-\sum_{i=1}^{8}\left(x_i - \frac{1}{\sqrt{8}}\right)^2\right)$$

$$f_2(x_1,...,x_8) = 1 - exp\left(-\sum_{i=1}^{8}\left(x_i + \frac{1}{\sqrt{8}}\right)^2\right)$$

where $-2 \le x_i < 2$

The Pareto optimal front is all the points on the line defined by

$$x_1 = x_2 = \ldots = x_8 \wedge \frac{-1}{\sqrt{8}} \le x_1 \le \frac{1}{\sqrt{8}}$$



Formal Definitions

- A multi-objective optimization problem (MOP) can be written as

$$\begin{aligned} \min F(\boldsymbol{x}) &= [f_1(\boldsymbol{x}) \quad f_2(\boldsymbol{x}) \quad \cdots \quad f_M(\boldsymbol{x})] \\ subject\ to\ \ G(\boldsymbol{x}) &= [g_1(\boldsymbol{x}) \quad g_2(\boldsymbol{x}) \quad \cdots \quad g_J(\boldsymbol{x})] \geq 0 \\ H(\boldsymbol{x}) &= [h_1(\boldsymbol{x}) \quad h_2(\boldsymbol{x}) \quad \cdots \quad h_K(\boldsymbol{x})] = 0 \\ x_i^{(L)} &\leq x_i \leq x_i^{(U)} \end{aligned}$$

- $\boldsymbol{x} = (x_1, x_2, \cdots, x_N)^T$ is the N dimensional decision variable,
- M is the number of objectives f_i,
- J inequality and K equality constraints,
- $x_i^{(L)}$ and $x_i^{(U)}$ are respectively the lower and upper bound for each decision variable x_i.

- For an MOP, M must be greater than unity
- Based on the constraints, a MOP can be classified into one of the following classes

J	K	$x_i^{(L)}$	$x_i^{(U)}$	***Type of MOP***
0	0	$-\infty$	∞	Unconstrained MOP
0	0	C_1	C_2	Bound constrained MOP
> 0	0	x	x	Inequality constrained MOP
0	> 0	x	x	Equality constrained MOP

where C_1 and C_2 are some constants.

- For the sake of simplicity, we use S denotes the feasible decision space. The feasible decision space is the region which satisfies the constrained in *decision space*

- By using this convention, an MOP can be rewritten as

$$\min_{\boldsymbol{x}\in S}[f_1(\boldsymbol{x}) \quad f_2(\boldsymbol{x}) \quad \cdots \quad f_M(\boldsymbol{x})]$$

Dominance relation

- Definition
  - A solution $\boldsymbol{x}_1$ is said to dominate another solution $\boldsymbol{x}_2$ ($\boldsymbol{x}_1 \preccurlyeq \boldsymbol{x}_2$), if and only if both of the following conditions are true:
    - The solution $\boldsymbol{x}_1$ is no worse than $\boldsymbol{x}_2$ in all objectives
    - The solution $\boldsymbol{x}_1$ is strictly better than $\boldsymbol{x}_2$ in at least one objective

- Mathematically,

$$\boldsymbol{x}_1 \preccurlyeq \boldsymbol{x}_2 \text{ if and only if } \begin{cases} f_i(\boldsymbol{x}_1) \leq f_i(\boldsymbol{x}_2) \; \forall i \in 1, \ldots, M \\ \exists j \in 1, \ldots, M \; | f_j(\boldsymbol{x}_1) < f_j(\boldsymbol{x}_2) \end{cases}$$

Non-dominated Set

- Given a set of solutions, we can perform all possible pairwise comparisons and find the solutions which are not dominated by any solution in the set
- Definition - Non-Dominated Set
  - Among a set of solutions P, the non-dominated set of solutions P' are those that are not dominated by any member of the set P
- Definition - Pareto Optimal Set (in decision space)
  - The set of solutions that are non-dominated in the feasible objective space
- Definition - Pareto Optimal Front (in objective space)
  - The set of non-dominated solutions in the feasible objective space

Classical Approaches

- Generating Pareto optimal solutions plays an important role in multi-objective optimization. The term vector optimization is sometimes used to denote the problem of identifying the Pareto optimal set
- Classical approaches often solve the MOP by scalarization. It means converting the MOP into a single or a family of SOP with a real-valued objective function
- We are interested in generating Pareto optimal solutions (POS)
- A-Posteriori Methods
  - Generate the Pareto optimal set (or a part of it)
  - Present it to the decision maker (DM)
  - Let the DM select one

Weighted Sum Method (WSM)

- This method scalarizes MOP into SOP by multiplying each objective with a predefined weight

$$\min_{x \epsilon S} \sum_{m=1}^{M} w_m f_m(\boldsymbol{x})$$

  where w_m is the weight of the m-th objective function. The weight coefficients w_m must be real and positive. It is usual practice that weight are normalized, such that $\sum_{m=1}^{M} w_m = 1$

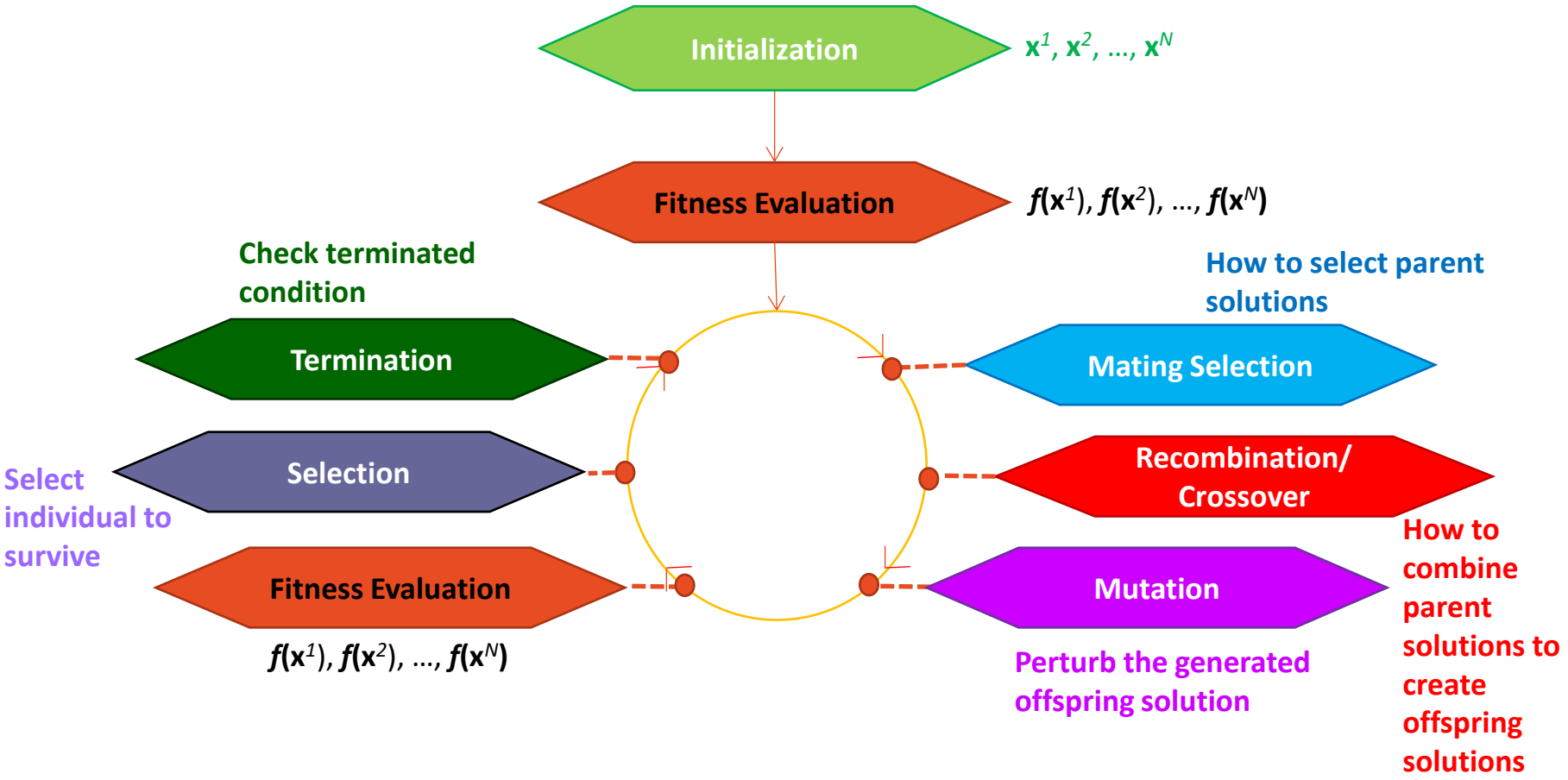
- To use WSM as a posteriori method, a set of weight vectors has to be specified. The correlation and some nonlinear effects between weights and MOP are not easy to be understood
- The advantage of WSM is that it is relatively simple and easy to use
- Evenly distributed weight vectors do not necessary produce an evenly distributed representation of the Pareto optimal set and some of the Pareto optimal solutions may fail to be found.

Multi-objective Evolutionary Algorithms (MOEA)

- Evolutionary Multi-objective Optimization (EMO) is an application of evolutionary algorithm to solve multi-objective optimization problem
- MOEAs are used to approximate the Pareto-optimal front of multi-objective problems
- Many MOEAs have been proposed in the literature
  - Multi-objective Genetic Algorithm (MOGA)
  - Non-Dominated Sorting Algorithm II (NSGAII)
  - Incrementing Multi-objective Evolutionary Algorithm (IMOEA)
  - Multiobjective Evolutionary Algorithm based on Decomposition (MOEA/D)
- These algorithms mainly different in, e.g., Fitness assignment scheme; Diversity preservation; Sharing and Elitism strategies

Tan, K. C., Khor, E. F. and Lee, T. H. *Multiobjective Evolutionary Algorithms and Applications*, Springer-Verlag, United Kingdom, 2005.

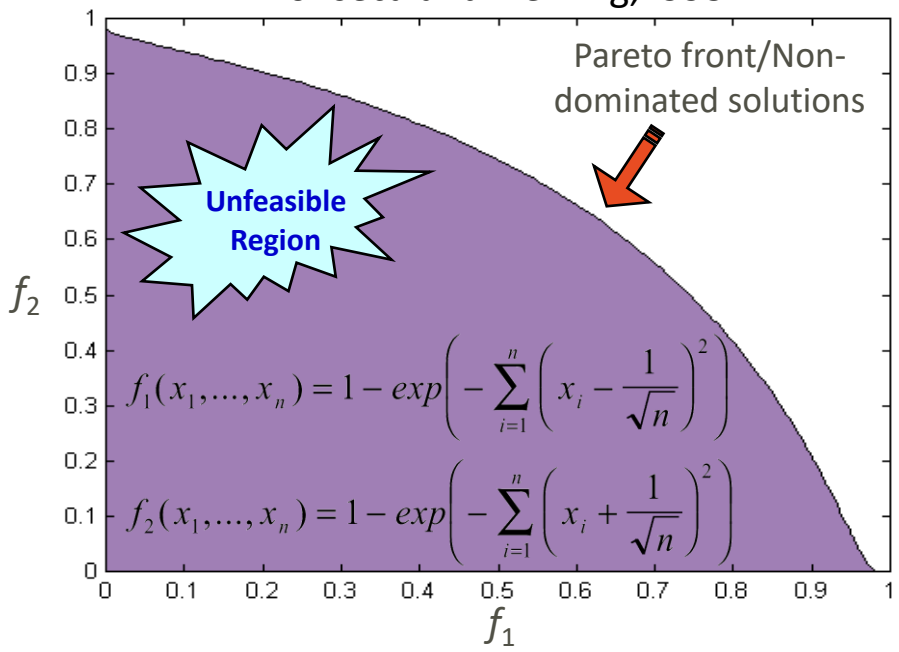
Multi-objective Evolutionary Algorithms (MOEA)



Multi-objective Evolutionary Algorithms (MOEA)

- EMO needs to address several issues
  - ✓ Fitness assignment, preference
  - ✓ Good spread, uniform distribution, minimum proximity
  - ✓ Many objectives, constraints
  - ✓ Noise, dynamic landscape, robust optimization…

Fonseca and Fleming,1995

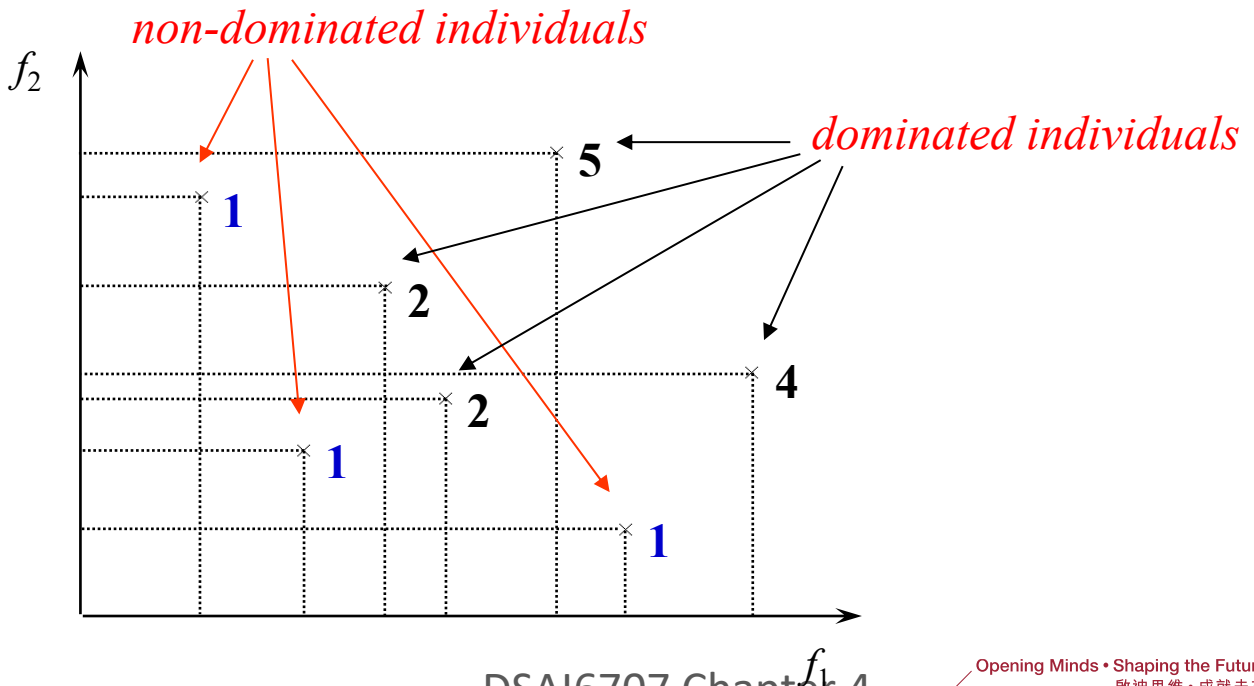


- EA is a powerful tool for solving MO optimization problems
  - ✓ Population-based, and capable of searching for the global trade-off
  - ✓ Robust and applicable to a wide range of problems
  - ✓ Capable of handling discontinuous and multi-dimensional problems

- The *Pareto ranking* assigns the same *smallest cost* for all *non-dominated individuals*, while the *dominated individuals* are ranked according to how many individuals in the population dominating them

- The rank of an individual x in a population can be given by $rank(\boldsymbol{F}_x) = 1 + q_x$, where q_x is the number of individuals dominating the individual $\boldsymbol{F}_x$ expressed in the objective domain.

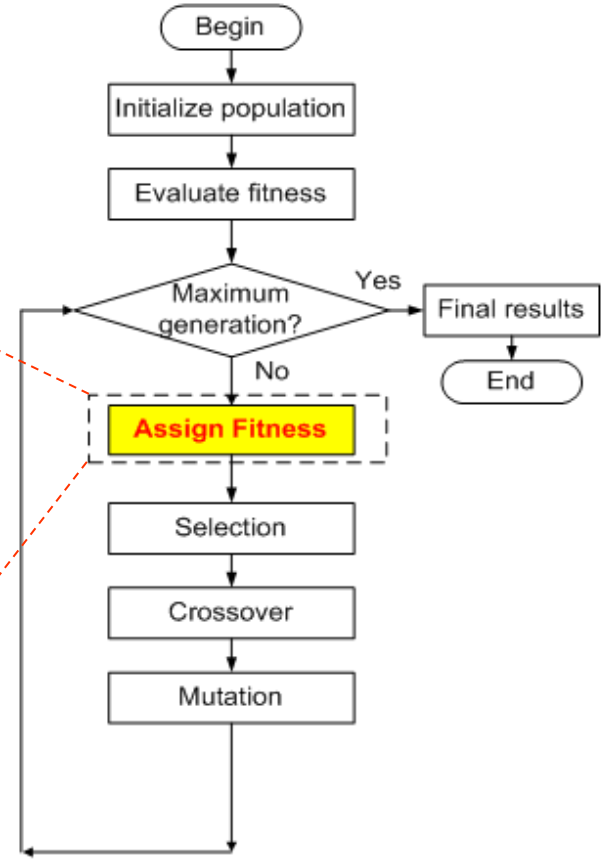
Pareto optimal ranking



What to change in a Simple GA for MO optimization?

Modify the Fitness Computation:

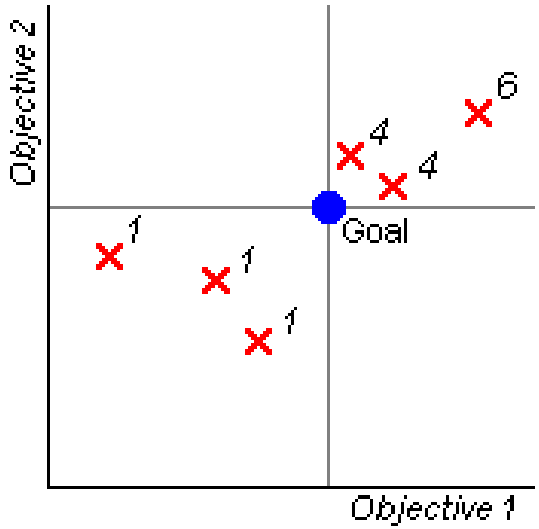
- *Emphasize non-dominated solutions for convergence (via Pareto ranking)*
- *Emphasize less crowded solutions for diversity (via sharing function)*



why sharing is needed for MO optimization?

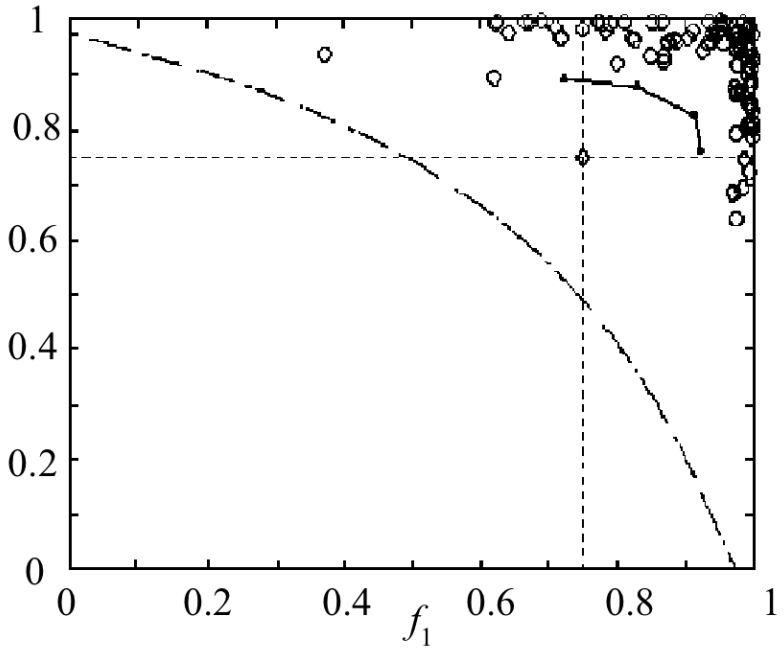
DECISION SUPPORT

- In design optimization, a set of specification or design requirements are often given in a-priori
- Goal (MOGA)
  - ✓ Desired value for each objective
  - ✓ Can be used to specify practical design specification/requirements
  - ✓ Individuals that satisfy the goal setting have a lower (better) rank



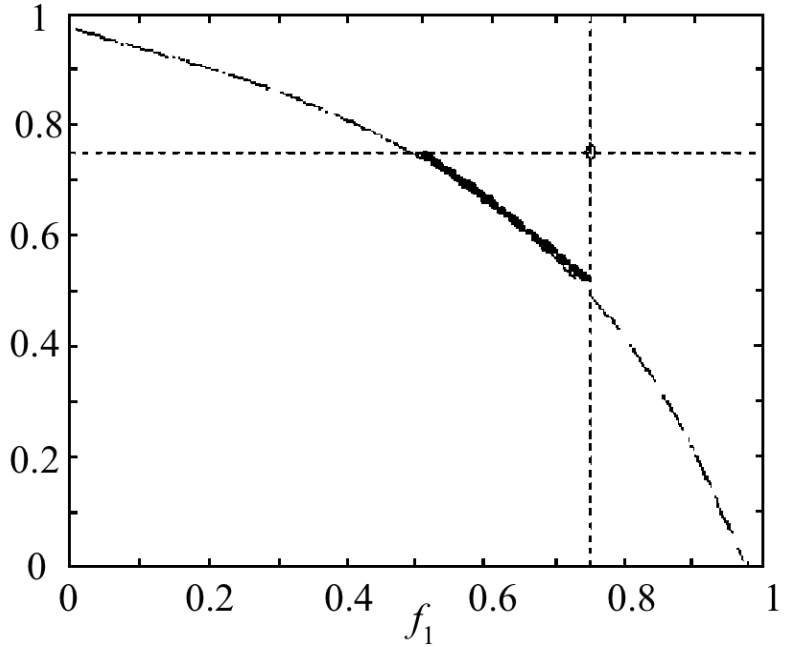
Tan, K. C., Khor, E. F., Lee, T. H. and Sathikannan, R., 'An evolutionary algorithm with advanced goal and priority specification for multi-objective optimization', *Journal of Artificial Intelligence Research,* vol. 18, pp. 183-215, 2003.

MOEA minimization with a feasible goal setting



h Individuals with rank = 1
o Individuals with rank > 1
◊ goal = (0.75,0.75)

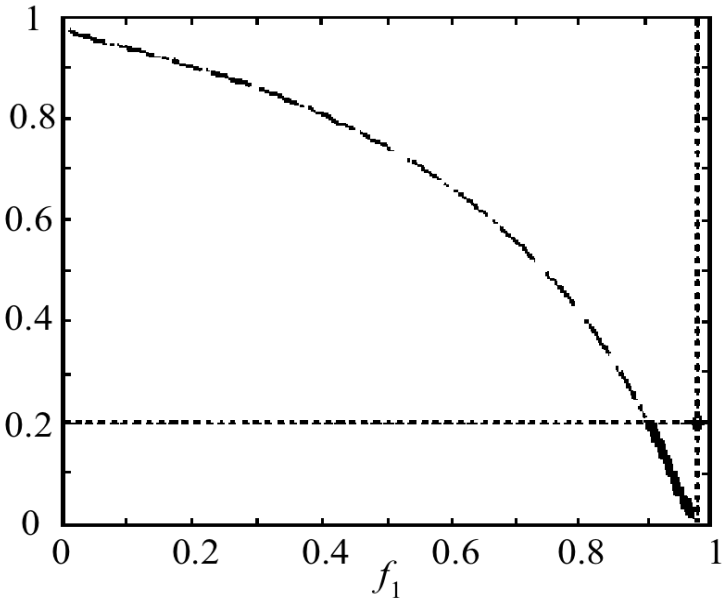
Gen = 5



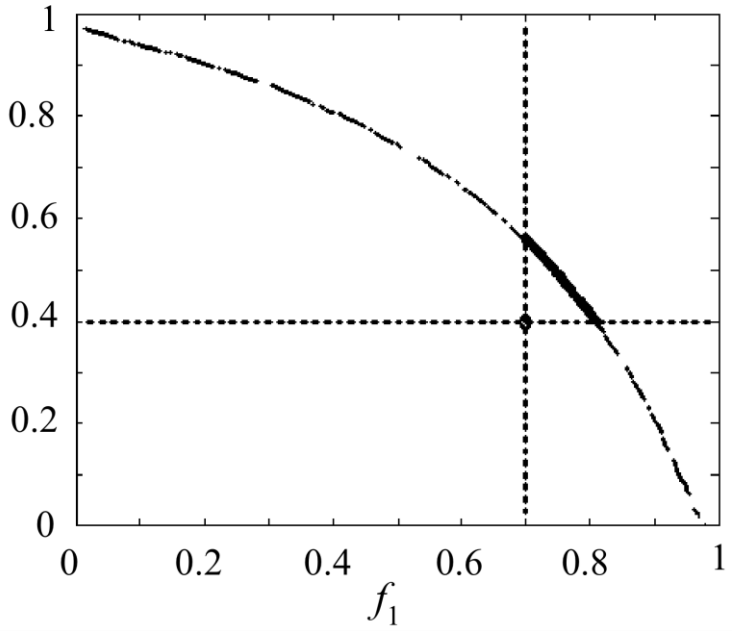
h Individuals with rank = 1
o Individuals with rank > 1
◊ goal = (0.75,0.75)

Gen = 70

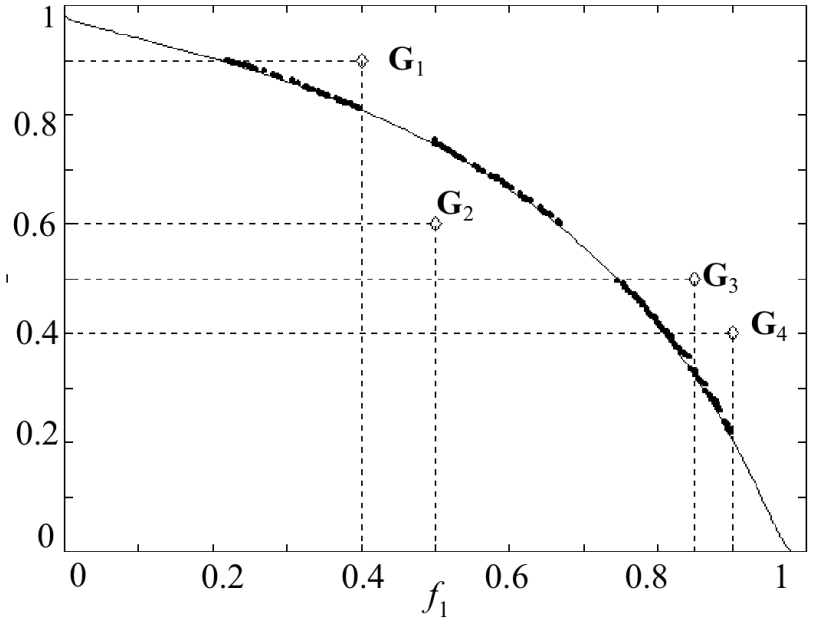
Feasible but extreme goal setting



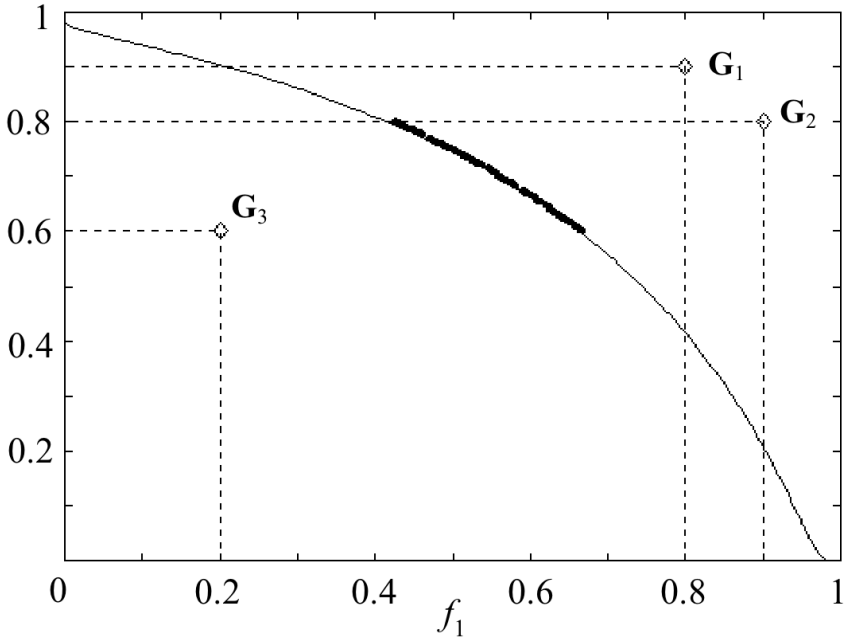
Unfeasible goal setting



Logical "OR" and "AND" connectives among goals



$(\mathbf{G}_1 \cup \mathbf{G}_2 \cup \mathbf{G}_3 \cup \mathbf{G}_4)$

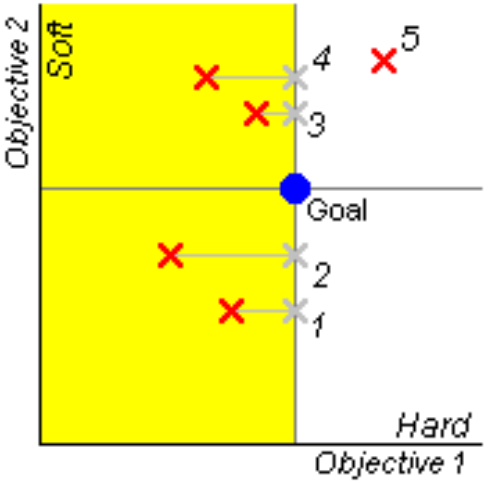


$(\mathbf{G}_1 \cap \mathbf{G}_2 \cap \mathbf{G}_3)$

- Priority/Preference
  - ✓ Assign relative importance of each objective for practical applications
  - ✓ Among strings of equal rank, priority can be used to determine superiority
  - ✓ Example: Stability in control system design



- Soft/hard Objectives
  - ✓ Soft: Always considered in the evolution
  - ✓ Hard: Considered while goal is not satisfied
  - ✓ Example: SS error and actuator saturation

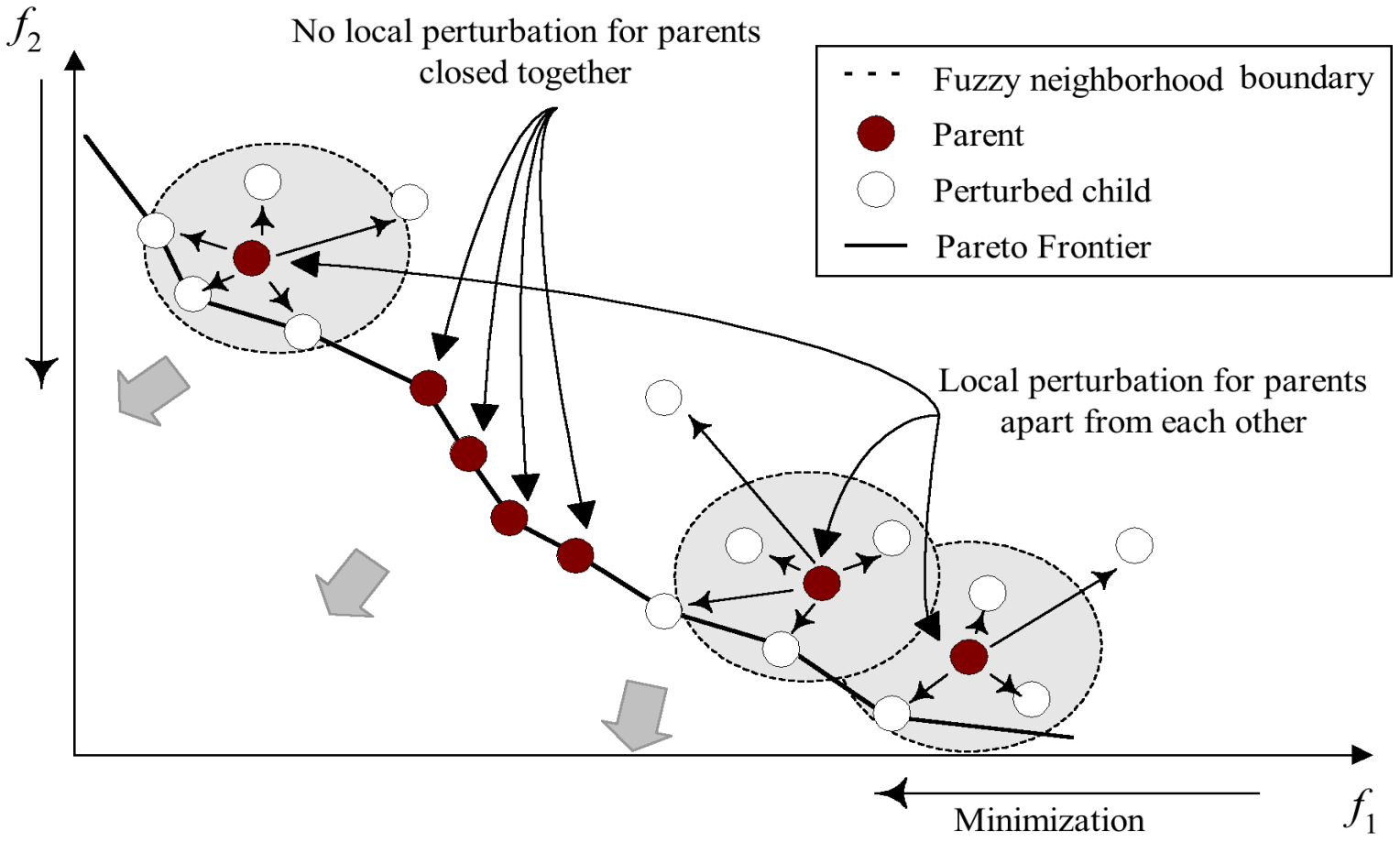


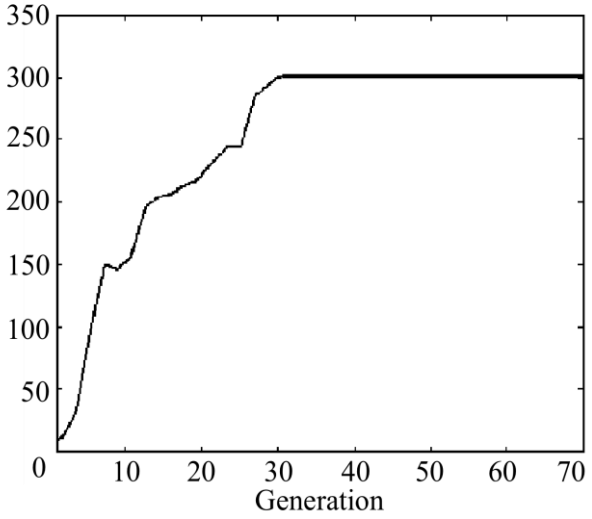
Dynamic Population

- ✓ Compared with SO problems, MO optimization often requires a larger population size in order to cover the trade-off surface
- ✓ Population size can be changed according to the population distribution at each generation
- ✓ Generally, we can start with a small population for initial search. Subsequently increase or decrease the population size according to current Pareto front in the evolution process
- ✓ The increased individuals can be obtained via local fine-tuning by generating additional "good" individuals to fill up gaps or discontinuities in the current Pareto front

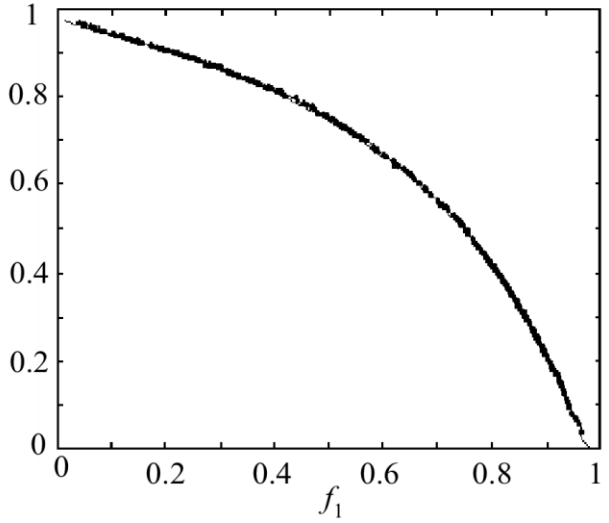
Tan, K. C., Lee, T. H. and Khor, E. F., 'Evolutionary algorithm with dynamic population size and local exploration for multiobjective optimization', *IEEE Transactions on Evolutionary Computation*, vol. 5, no. 6, pp. 565-588, 2001.

Local Perturbation

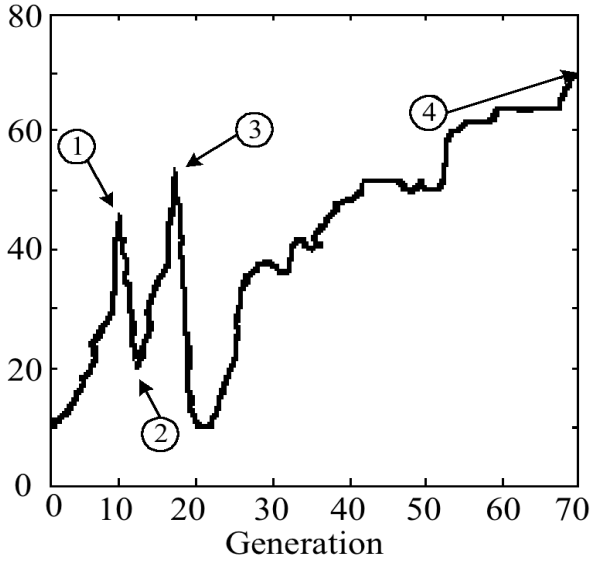


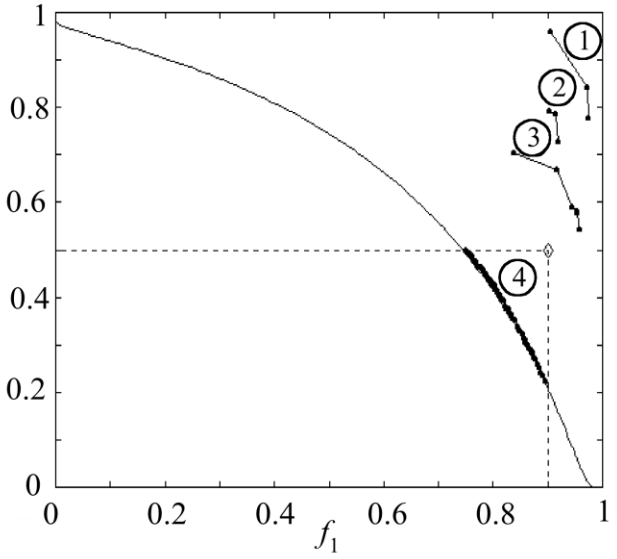


Population size versus generation



Population distribution





JOURNALS

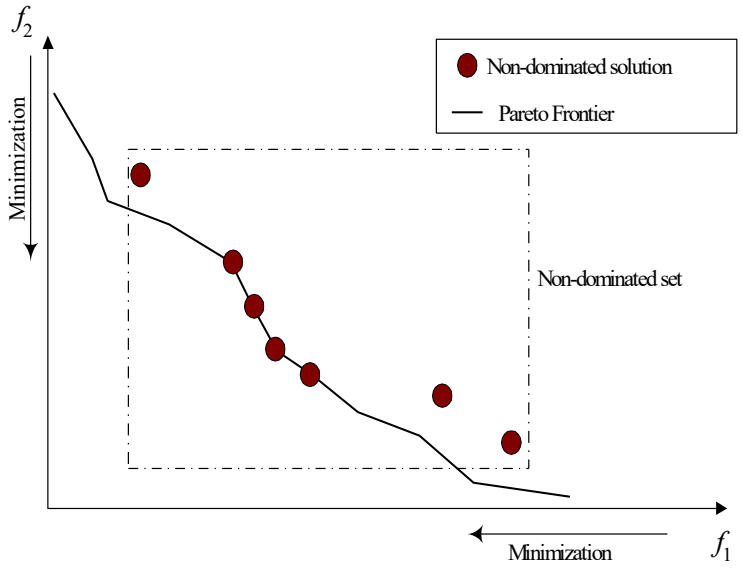
- **IEEE Transactions on Evolutionary Computation**
- Evolutionary Computation: MIT Press
- IEEE Transaction on Cybernetics
- Expert Systems with Applications
- Studies in Computational Intelligence
- Applied Soft Computing Journal
- European Journal of Operational Research
- Energy
- Soft Computing
- Computers and Operational Research
- Engineering Applications of Artificial Intelligence
- Information Science
- Memetic Computing
- Annals of Operational Research
- Artificial Intelligence Review
- Natural Computing
- International Journal of Innovative Computing, Information and Control
- Engineering Optimization
- **IEEE Computational Intelligence Magazine**
- Computational Optimization and Applications
- Transactions on Electronic, Information and Systems
- International Journal of Approximate Reasoning
- International Journal of Computational Intelligence and Applications
- Neurocomputing
- International Journal of Computational Intelligence Research
- Soft Computing
- Journal of Computer Science and Technology
- Journal of Scheduling
- Journal of global optimization
- Journal of Engineering Design
- International Transactions in Operational Research
- International Journal of Computational Intelligence System
- Journal of the Operational Research Society
- European Transactions on Electrical Power
- Journal of Intelligence Manufacturing
- Journal of System Architecture
- Frontiers of Computer Science

SOME PERFORMANCE METRICS

- Generational Distance (GD) (Veldhuizen, 1999)
  Measures how far the evolved solution set is from the true Pareto front.

- Spacing (S) (Schott, 1995)
  Measures how evenly the evolved solutions distribute itself.

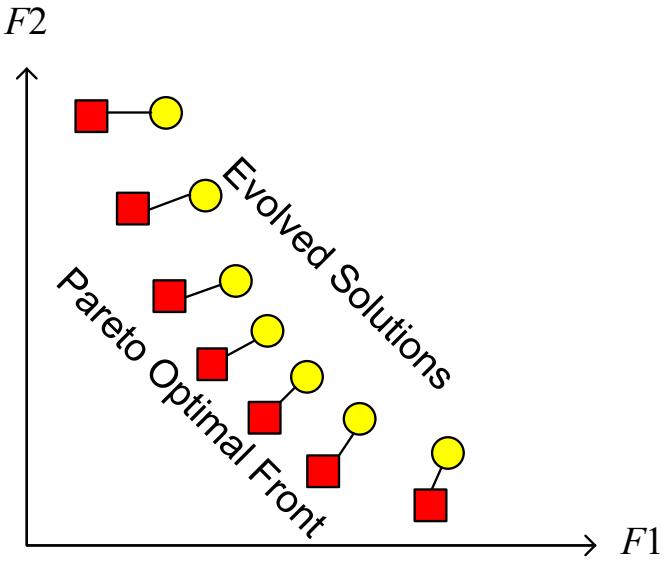
- Maximum Spread (MS) (Zitzler, 1999)
  Measures how well the true Pareto front is covered by the evolved solution set.

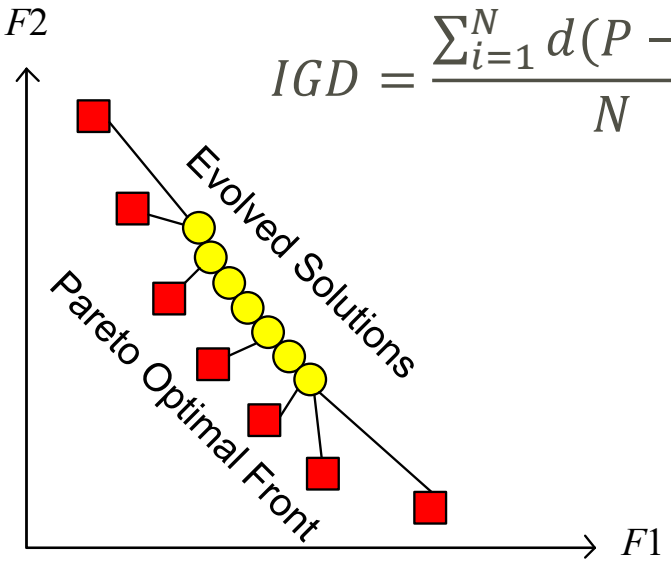


- Hyper-Volume Ratio (HVR) (Veldhuizen, 1999)
  Calculates the volume covered by the evolved solutions.

PERFORMANCE ASSESSMENT

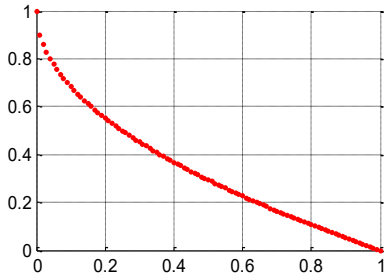
- **Inverted Generational Distance (IGD)** (M. A. Villalobos-Arias, 2005)
  - ✓ Measure the proximity as well as diversity between the obtained Pareto front and the Pareto Optimal front
  - ✓ The Euclidean distance is measured from each obtained solution to the optimal solutions set



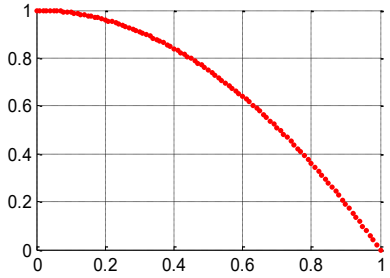


$$IGD = \frac{\sum_{i=1}^{N} d(P - P^*)_i}{N}$$

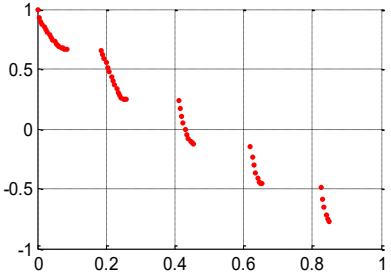
A FEW TEST PROBLEMS



ZDT1

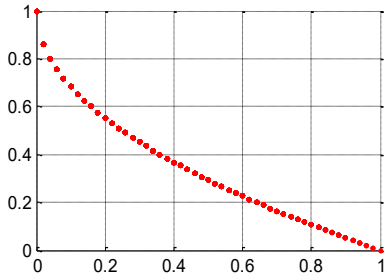


ZDT2

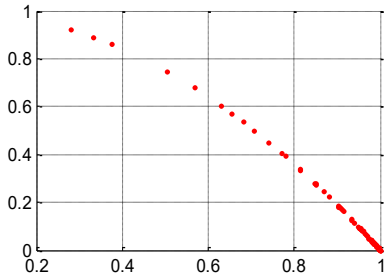


ZDT3

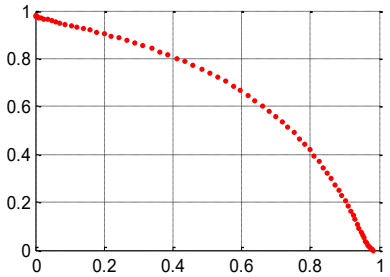
	Test Problem	Features
1	ZDT1	Pareto front is convex.
2	ZDT2	Pareto front is non-convex.
3	ZDT3	Pareto front consists of several noncontiguous convex parts.
4	ZDT4	Pareto front is highly multi-modal where there are 21^9 local Pareto fronts.
5	ZDT6	The Pareto optimal solutions are non-uniformly distributed along the global Pareto front. The density of the solutions is low near the Pareto front and high away from the front.



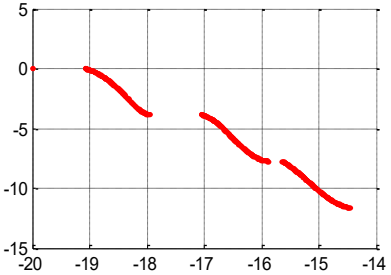
ZDT4



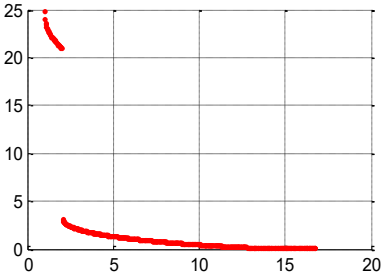
ZDT6



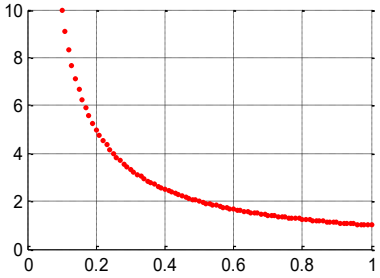
FON



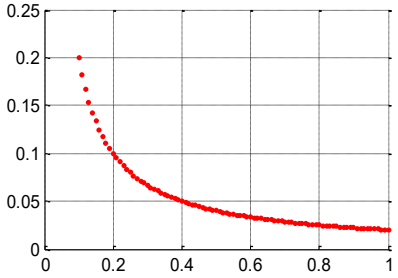
KUR



POL



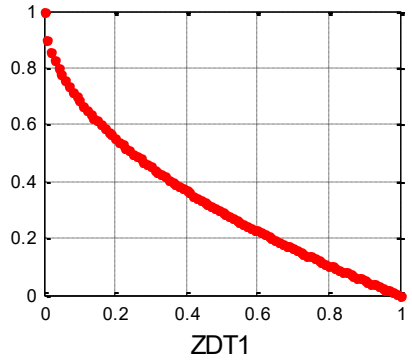
TLK

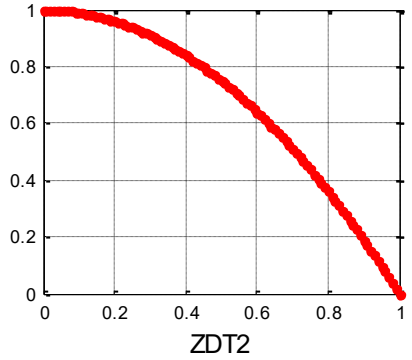


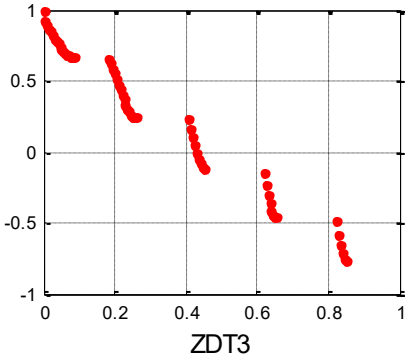
TLK2

	Test Problem	Features
6	FON	Pareto front is non-convex.
7	KUR	Pareto front consists of several noncontiguous convex parts.
8	POL	Pareto front and Pareto optimal solutions consist of several noncontiguous convex parts.
9	TLK	Noisy landscape.
10	TLK2	Non-stationary environment.

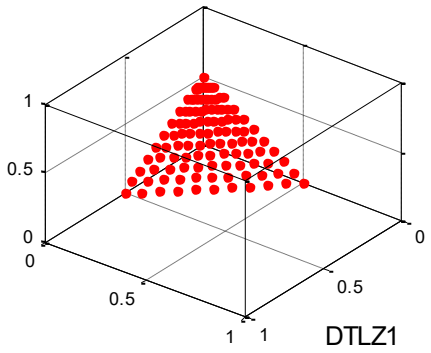
SOME TEST PROBLEMS

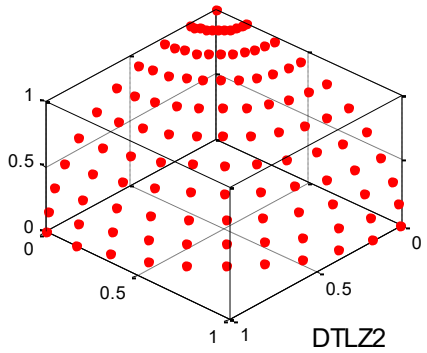


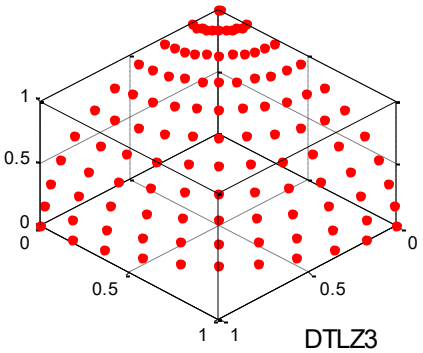




	Problem (variables)	Remarks
1	ZDT1 (100)	Convex Pareto front
2	ZDT2 (100)	Concave Pareto front
3	ZDT3 (100)	Non-continuous concave Pareto front
4	ZDT4 (10)	Convex Pareto front and highly multi-modality
5	ZDT6 (100)	Non-uniformity concave Pareto front
6	DTLZ1 (20)	High-dimensionality and linear Pareto optimal front
7	DTLZ2 (20)	High-dimensionality and spherical Pareto optimal front
8	DTLZ3 (20)	High-dimensionality and high multi-modality.



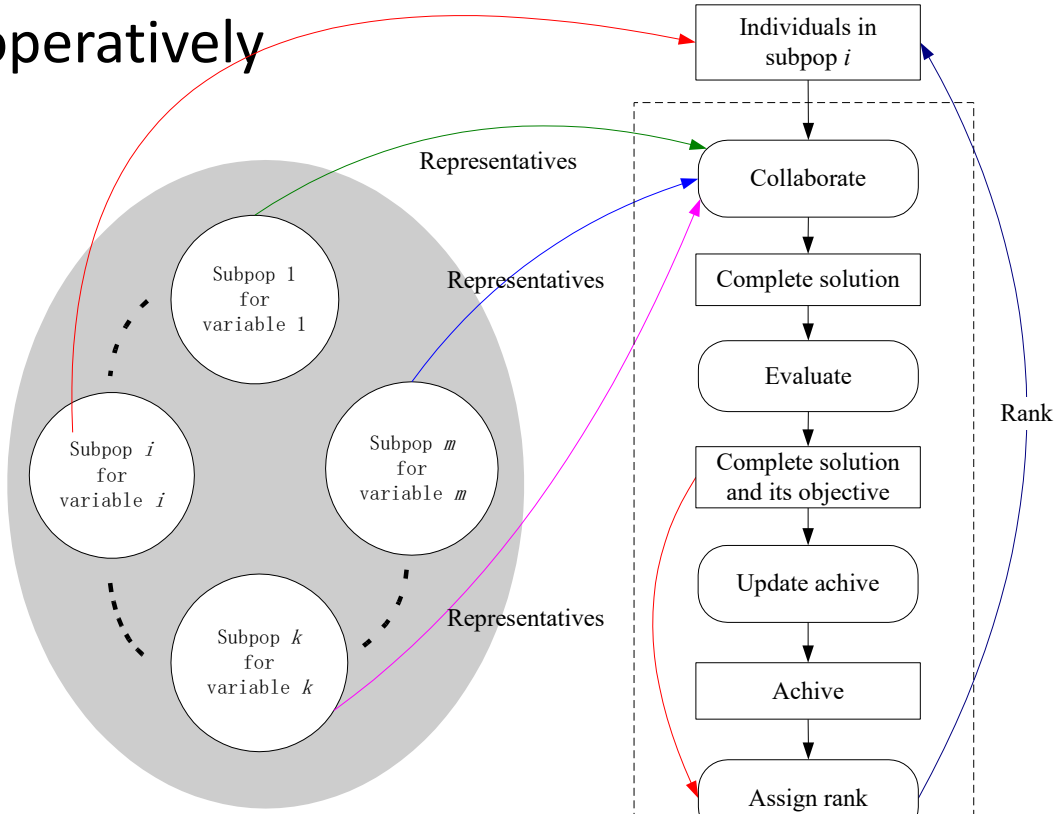




A COOPERATIVE CO-EVOLUTIONARY ALGORITHM

- Decompose a complex problem into smaller problems via co-evolving subpopulations cooperatively

  - ✓ A divide-and-conquer strategy
  - ✓ Each subpopulation evolves a different decision variable
  - ✓ Fitness is dependent on the collaboration between each subpopulation



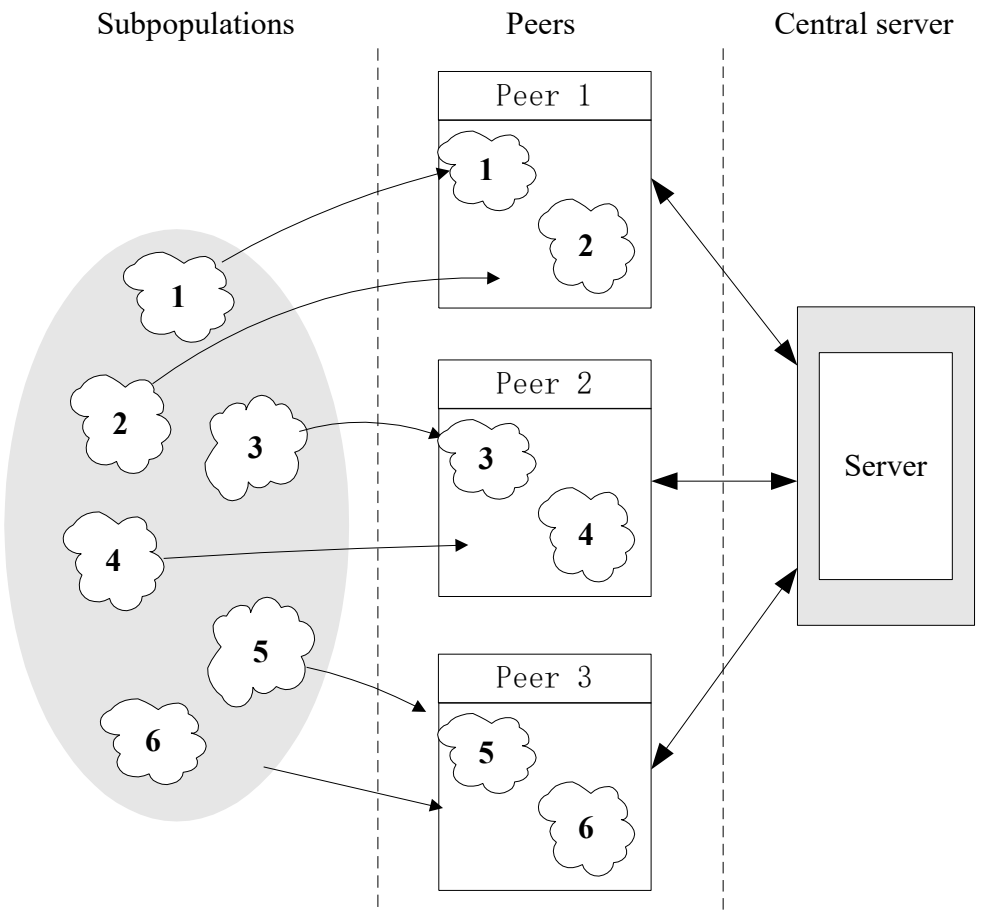
K. C. Tan, Y. J. Yang, C. K. Goh, "A distributed cooperative coevolutionary algorithm for multi-objective optimization", *IEEE Transactions on Evolutionary Computation*, Vol. 10, No. 5, pp. 527-549, 2006.

Cooperation Methods

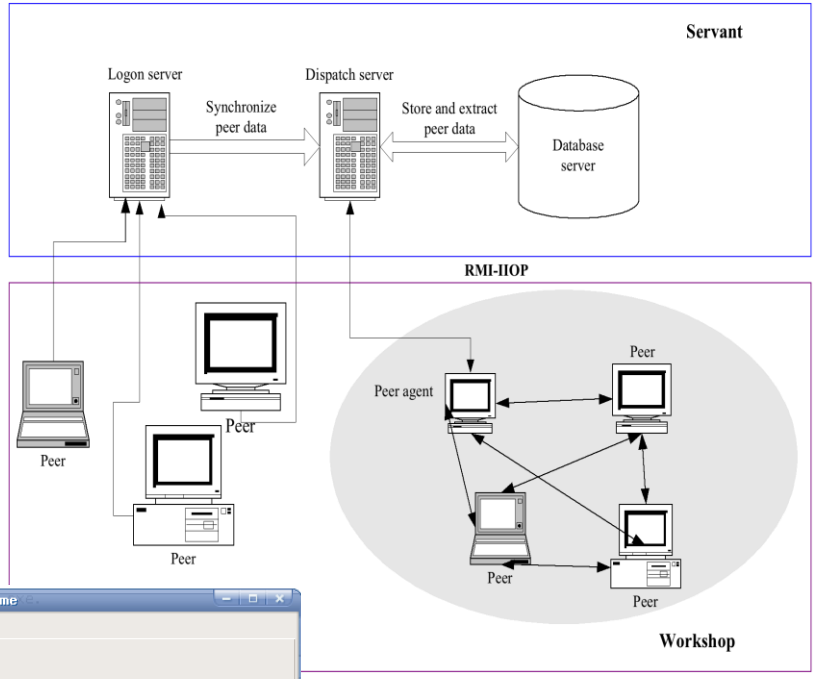
- The model of cooperation among subpopulations is an important issue in cooperative co-evolution
  - ✓ The simplest approach is to select the best individual as the representative. However, this approach tends to perform poorly for problems with high parameter inter-dependency
  - ✓ Alternatively, a random individual can be selected as the representative. However, this often results in a slower convergence rate
- Four different methods have been examined.
  - ✓ C1: The best individual is selected as representative
  - ✓ C2: The best and a random individuals are selected as representatives. The better solution resulting from the two co-operations is retained
  - ✓ C3: The best two individuals are selected as representatives. The better solution resulting from the co-operations with the best representative and either representative is retained
  - ✓ C4: The best two individuals are selected as the representatives. Cooperation is performed with either one of the representatives

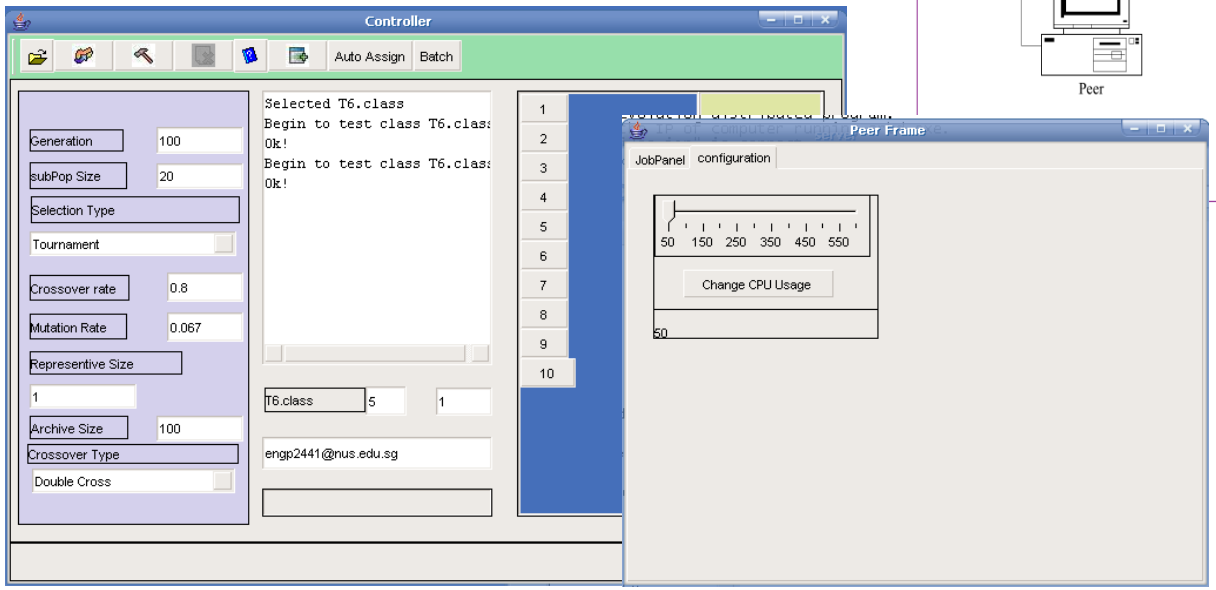
A Distributed CCEA

- Parallelization Strategy
  - ✓ Subpopulations are partitioned into a number of groups and assigned to peer computers
  - ✓ Indirect cooperation is achieved through the exchange of archive and representatives between peers and a central server
  - ✓ Peers are synchronized at fixed intervals to ensure better cooperation



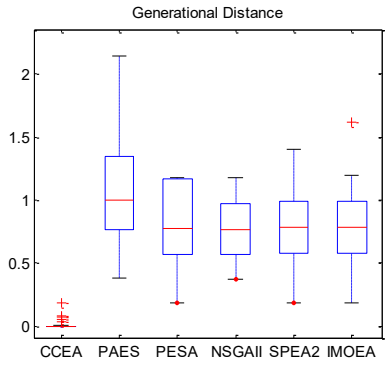
- The DCCEA is designed and embedded in a distributed computing framework named Paladin-DEC.
  - ✓ Developed upon the technology of Java 2 platform of Enterprise Edition (J2EE)
  - ✓ Exploit the inherent parallelism of evolutionary algorithms
  - ✓ Incorporates the features of robustness, security, and workload balancing



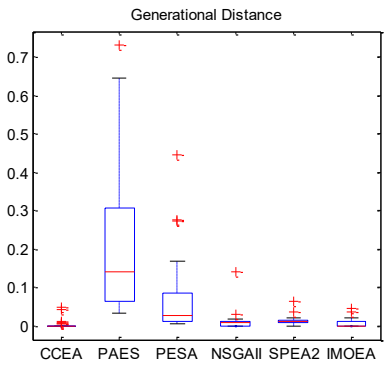


Comparative Results

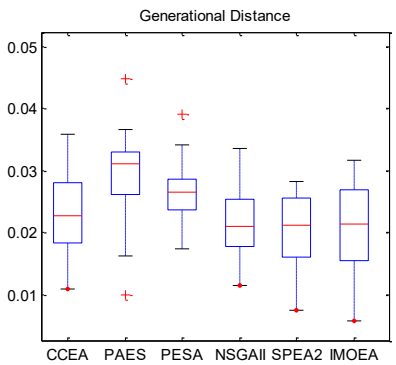
Generational Distance



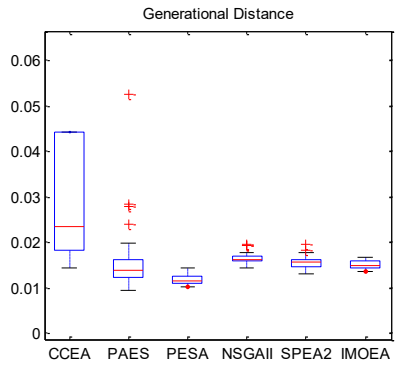
ZDT4



ZDT6



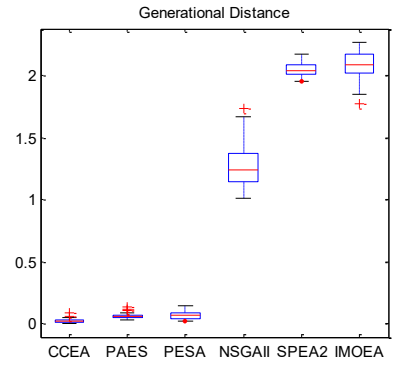
FON



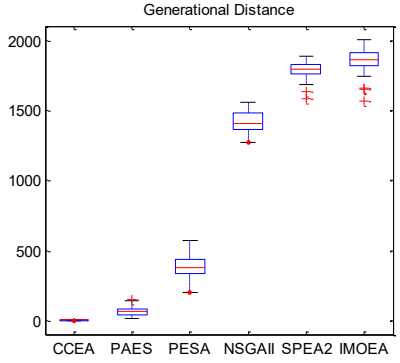
KUR

Observation:

- CCEA escapes from local optima of ZDT4 and DTLZ3
- CCEA scales well for high-dimensional MO problems
- CCEA performs less well for KUR due to the high parameter inter-dependency



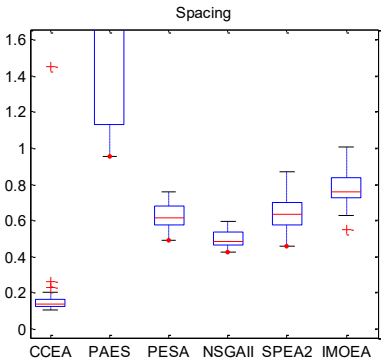
DTLZ2



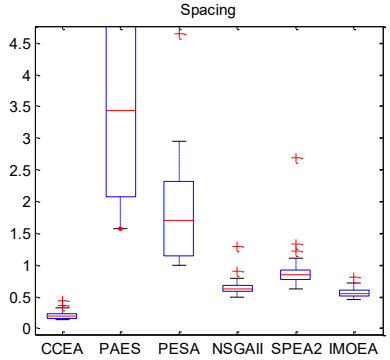
DTLZ3

Comparative Results

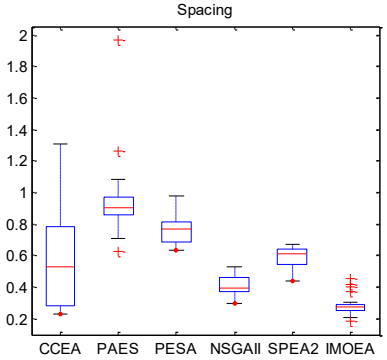
Spacing



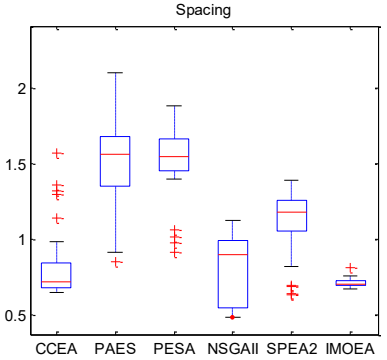
ZDT4



ZDT6



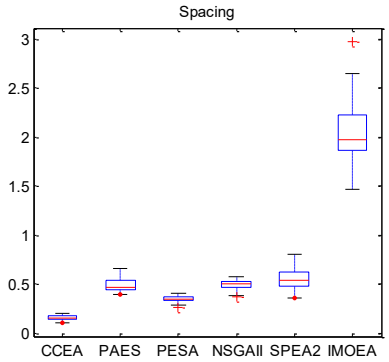
FON



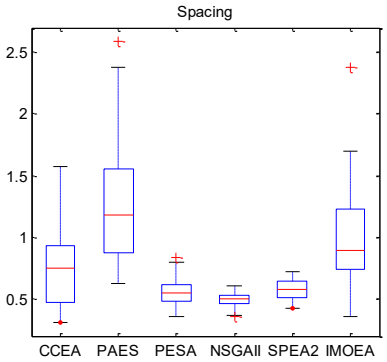
KUR

Observation:

- CCEA generally shows good performance in spacing
- CCEA handles the non-uniform distribution of ZDT6 well



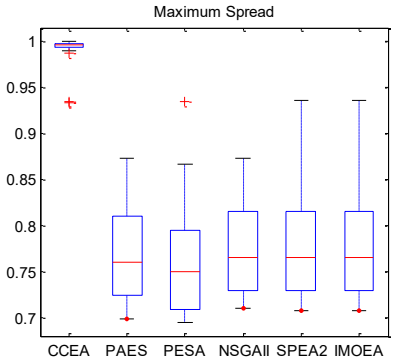
DTLZ2



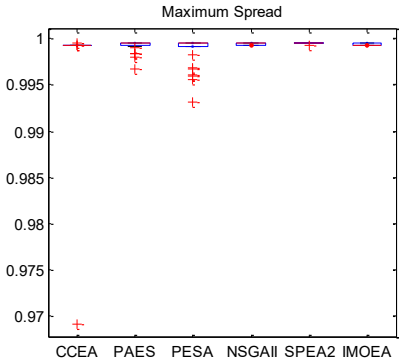
DTLZ3

Comparative Results

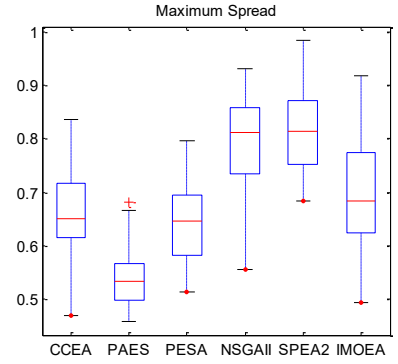
Maximum Spread



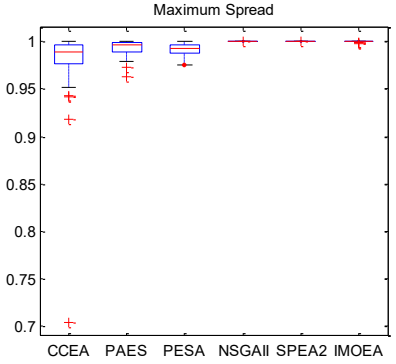
ZDT4



ZDT6



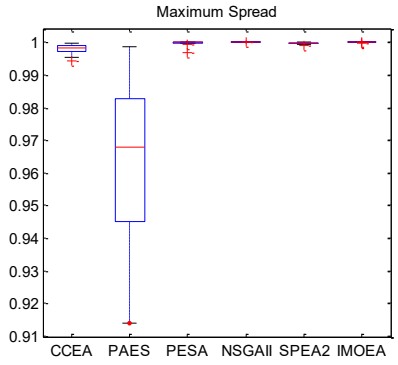
FON



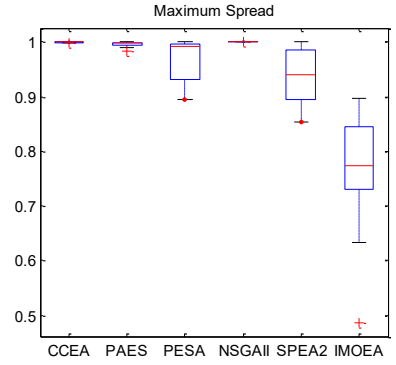
KUR

Observation:

- CCEA performs competitively as compared to other MOEAs



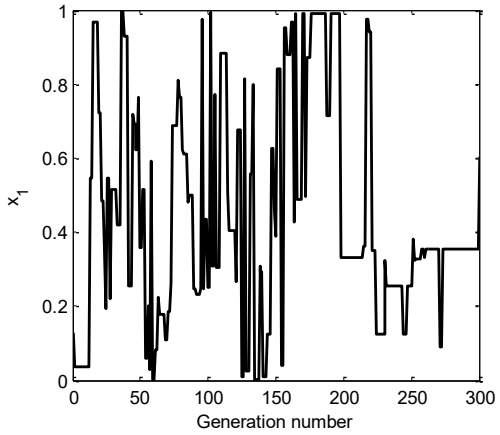
DTLZ2

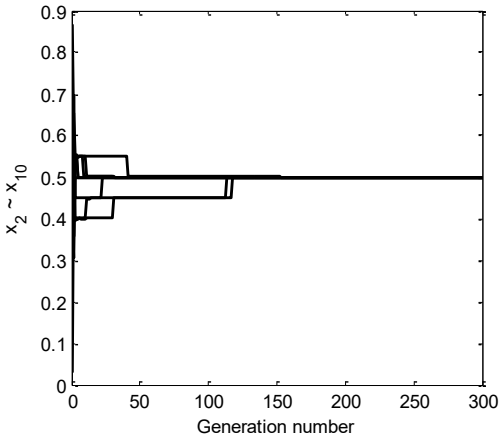


DTLZ3

Behaviors of CCEA

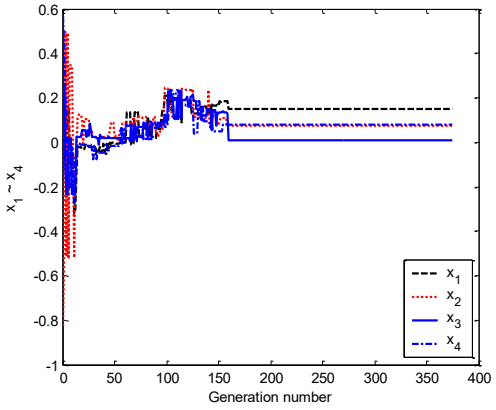
- ZDT4
  - ✓ Oscillations of x_1 help to sample the solutions along the Pareto front uniformly
  - ✓ Variables x_2 to x_{10} converge to the optimal location gradually

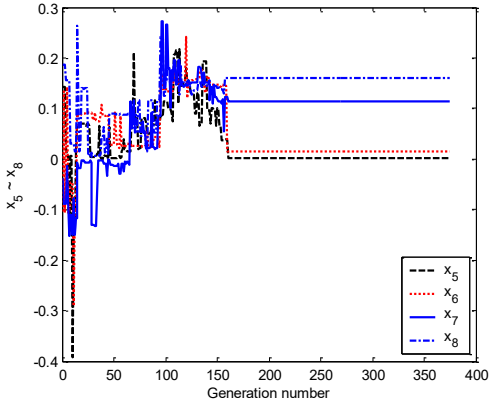




ZDT4

- FON
  - ✓ All the variables oscillate and converge in a similar pace
  - ✓ This behavior is related to the high inter-dependency among the parameters





FON

Effects of the Cooperation

Median GD for CCEA

Problem	C1	C2	C3	C4
ZDT1	4.13E-04	4.24E-04	4.04E-04	3.14E-01
ZDT2	4.05E-04	4.81E-04	6.35E-04	5.09E-01
ZDT3	1.45E-03	1.41E-03	1.27E-03	3.64E-01
ZDT4	6.98E-05	7.08E-05	1.48E-04	1.26E+00
ZDT5	1.27E-06	1.27E-06	1.26E-06	2.90E+00
ZDT6	4.89E-07	5.00E-07	4.86E-07	1.43E+00
FON	2.35E-02	2.32E-02	2.28E-02	2.13E-02
KUR	3.55E-02	2.31E-02	2.35E-02	1.76E-02
TLK	4.80E-01	5.23E-01	4.90E-01	5.34E-01
DTLZ2	8.57E-04	4.07E-01	2.08E-02	5.44E-01
DTLZ3	1.82E+00	2.24E+00	1.65E+00	5.82E+02

Median S for CCEA

Problem	C1	C2	C3	C4
ZDT1	0.1757	0.1771	0.1667	0.8240
ZDT2	0.1613	0.1553	0.1600	0.9328
ZDT3	0.2815	0.2867	0.2733	0.9952
ZDT4	0.1333	0.1436	0.1409	1.9624
ZDT5	0.7114	0.7125	0.7125	1.1676
ZDT6	0.1754	0.1822	0.1881	0.8174
FON	0.2687	0.3497	0.5291	0.2676
KUR	0.7919	0.7145	0.7154	0.6817
TLK	1.0905	1.0284	1.0135	1.0772
DTLZ2	0.1245	0.2658	0.1604	0.2929
DTLZ3	0.6638	0.8103	0.7525	0.7201

- C1 is the greediest method. It performs well for problems with low parameter inter-dependency, but gives poor results for those with high inter-dependency
- C4 is the least greedy approach. It performs well for problems with high parameter inter-dependency, but gives poor results for those with low inter-dependency
- C2 and C3 provide a balance between greedy search and diversity maintenance, which achieve generally good and robust performances

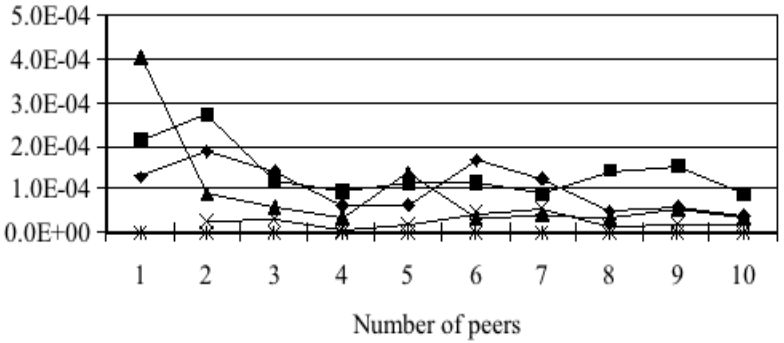
11 PCs in LAN

PC	Configuration CPU (MHz)/RAM (MB)
server	PIV 1600/512
peer 1	PIII 800/ 512
peer 2	PIII 800/ 512
peer 3	PIII 800/ 256
peer 4	PIII 933/384
peer 5	PIII 933/128
peer 6	PIV 1300/ 128
peer 7	PIV 1300/ 128
peer 8	PIII 933/ 512
peer 9	PIII 933/ 512
peer 10	PIII 933/256

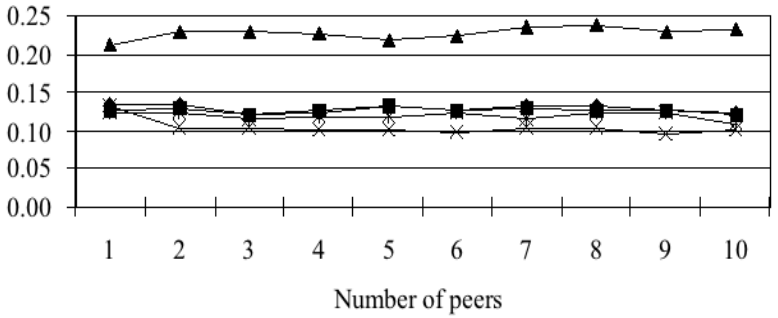
Configurations

Populations	Subpopulation size 20; archive size 100
Chromosome length	30 bits for each variable
Selection	Tournament selection
Crossover operator	Uniform crossover
Crossover rate	0.8
Mutation operator	Bit-flip mutation
Mutation rate	$2/L$, where L is the chromosome length
Number of evaluation	120,000
Exchange interval	5 generations
Sync. interval	10 generations

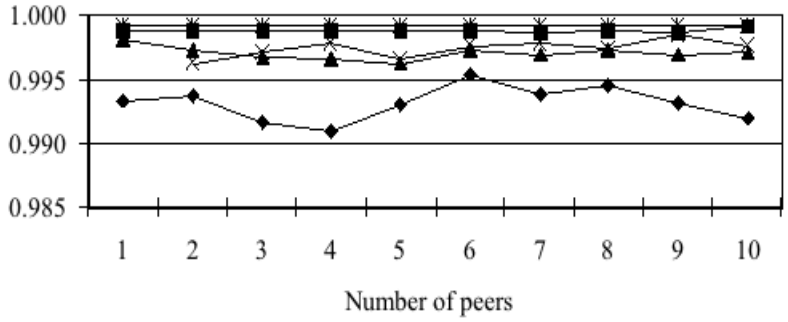
Generational Distance



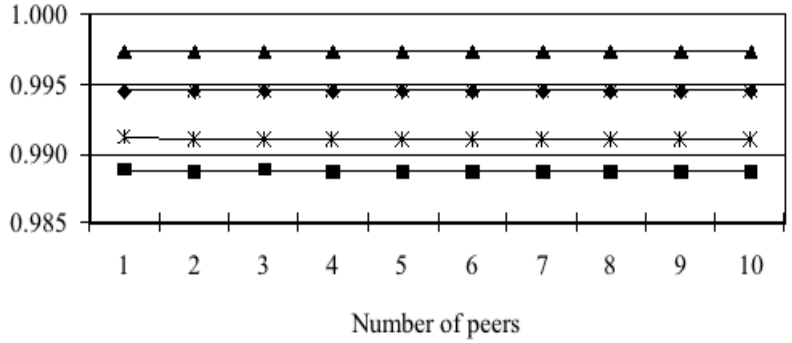
Spacing



Maximum Spread



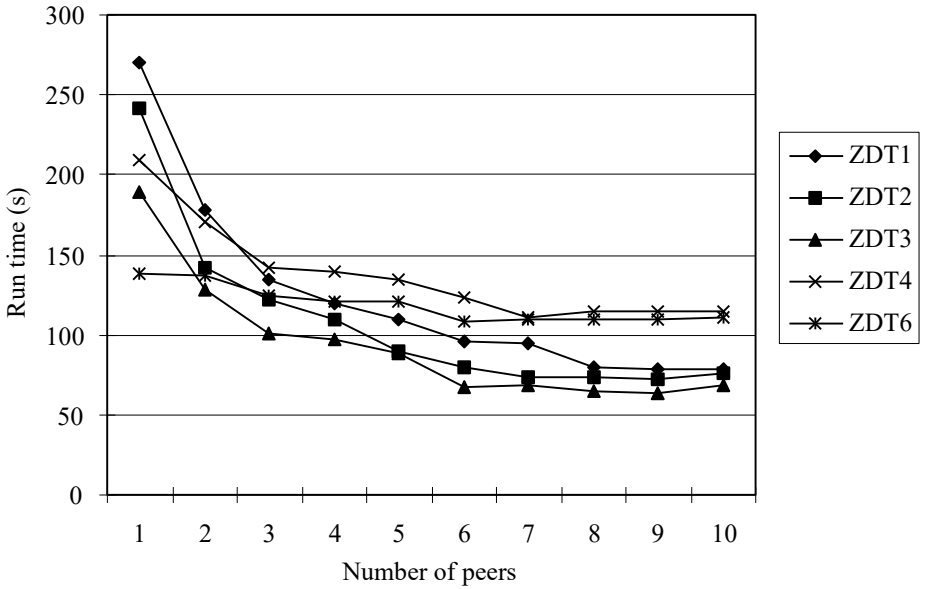
Hyper-volume Ratio



Speedup

Number of peers	ZDT1	ZDT2	ZDT3	ZDT4	ZDT6
2	1.52	1.70	1.47	1.23	1.01
3	2.01	1.99	1.88	1.47	1.11
4	2.25	2.21	1.95	1.50	1.14
5	2.48	2.69	2.15	1.56	1.14
6	2.81	3.03	2.83	1.70	1.29
7	2.87	3.32	2.77	1.88	1.25
8	3.38	3.27	2.92	1.82	1.26
9	3.46	3.36	2.96	1.83	1.26
10	3.46	3.18	2.79	1.82	1.25

Runtime



- Observation
  - ✓ Effective in reducing simulation runtime without sacrificing performance
  - ✓ Speedup achievable is more significant for large problems
  - ✓ The increase of communication cost counteracts the reduction of computation cost with increasing number of peers